Python Activity-Based Learning

Case Study 1: Attendance Recovery Computation

1. Problem Statement:

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach the required attendance percentage.

2. Objectives:

- Calculate the current attendance percentage.
- Use a loop to find the minimum number of future classes needed.
- Display the eligibility result.

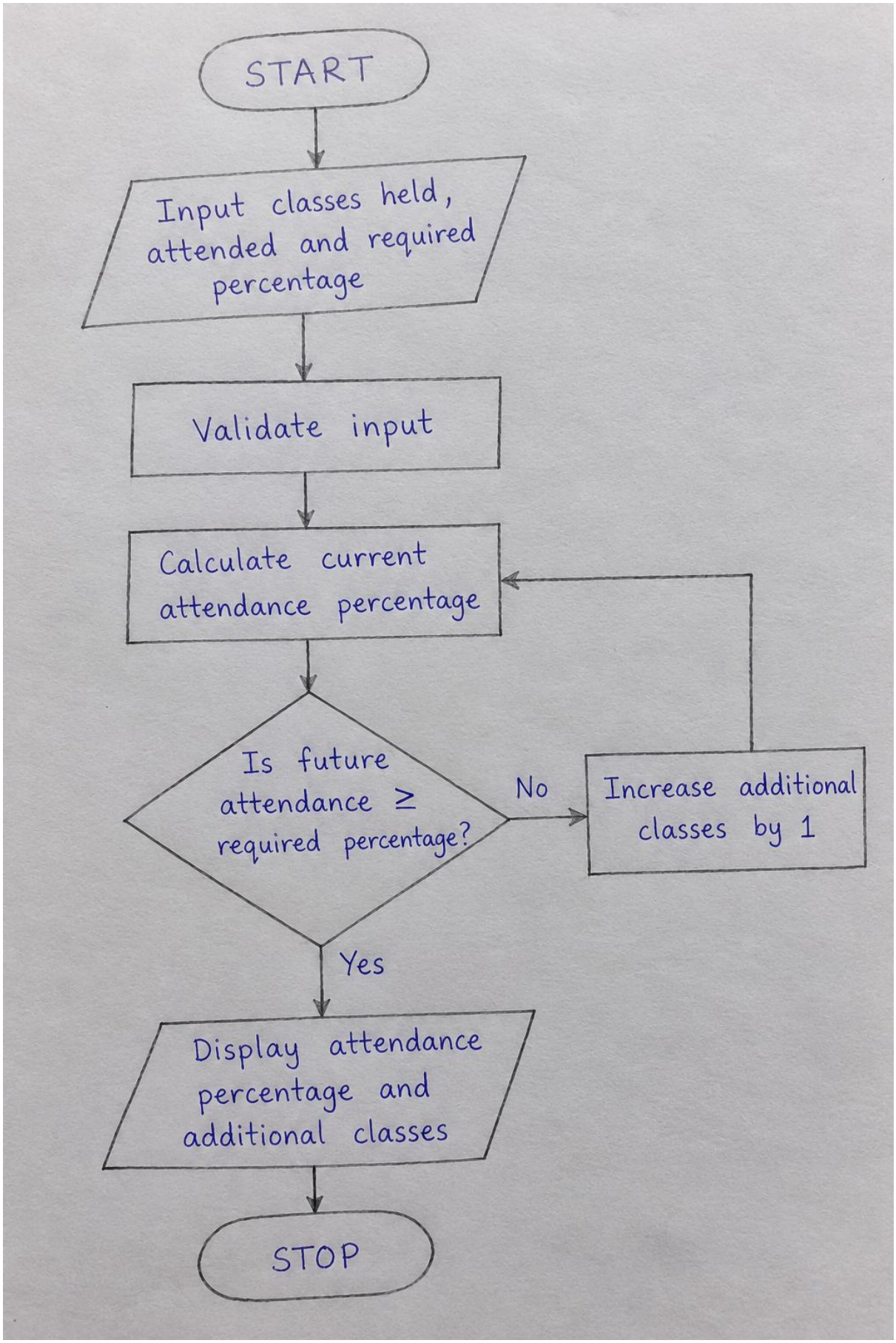
3. Assumptions:

- Total classes held must be greater than zero.
- Classes attended cannot be negative or greater than classes held.
- The student attends every future class.
- The required attendance percentage is between 0% and 100%.

4. Algorithm:

1. Start.
2. Input total classes held, classes attended, and required attendance percentage.
3. Validate the input values.
4. Calculate the current attendance percentage.
5. Set the number of additional classes to zero.
6. Use a while loop to check whether attendance reaches the required percentage.
7. Increase the additional class count until the target is reached.
8. Display current attendance and additional classes required.
9. Stop.

5. Flowchart:


START
Input classes held, attended and required percentage
Validate input
Calculate current attendance percentage
Is future attendance ≥ required percentage?
No
Increase additional classes by 1
Yes
Display attendance percentage and additional classes
STOP

6. Pseudocode:

```
START
INPUT held, attended, required

IF held <= 0 OR attended < 0 OR attended > held
   DISPLAY "Invalid input"
ELSE
   current = attended / held * 100
   future = 0

   WHILE (attended + future) / (held + future) * 100 < required
      future = future + 1

   DISPLAY current
   DISPLAY future
STOP
```

7. Python Source Code:

```
held = int(input("Classes held: "))
attended = int(input("Classes attended: "))
required = float(input("Required attendance percentage: "))

if (held <= 0 or attended < 0 or attended > held
      or required < 0 or required > 100):
   print("Invalid input")
else:
   current = attended / held * 100
   future = 0
```

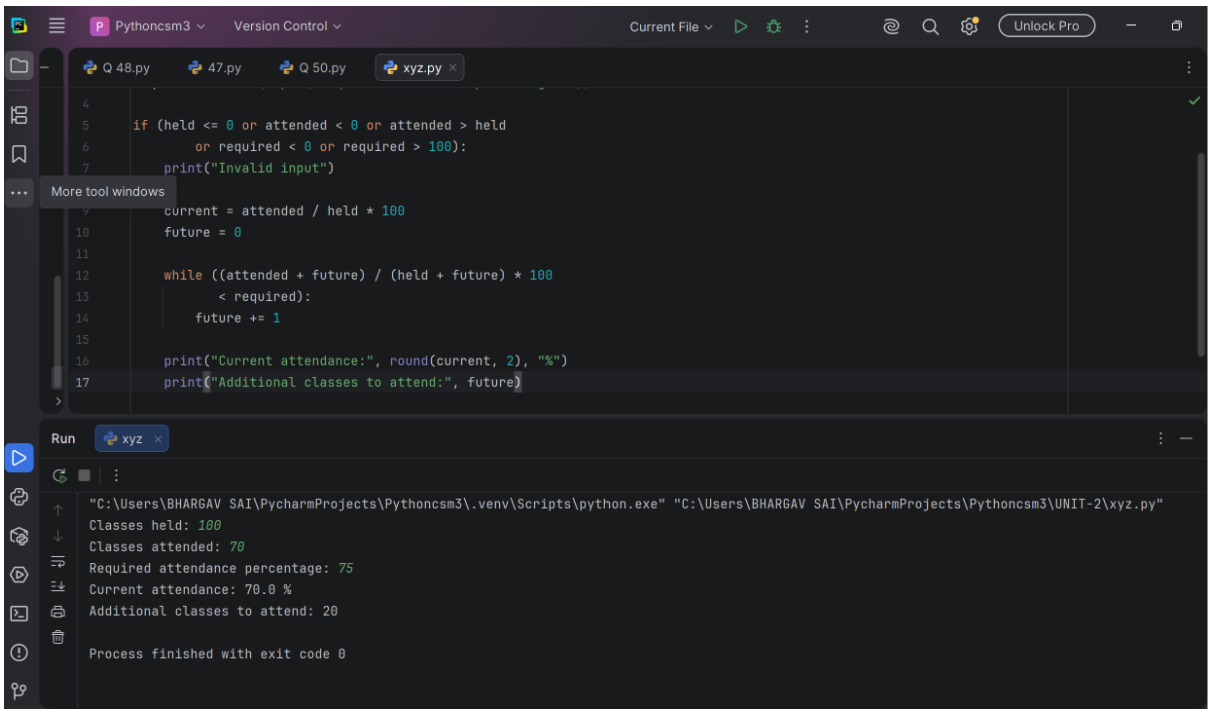
```
while ((attended + future) / (held + future) * 100
      < required):
   future += 1

print("Current attendance:", round(current, 2), "%")
print("Additional classes to attend:", future)
```

8. Test Cases:

Classes held	Attended	Required	Expected result
100	70	75%	Attend 20 more classes
50	45	75%	No additional classes needed
0	0	75%	Invalid input

9. Output:



10. Conclusion

Loops and conditional statements can be used to calculate attendance and determine the minimum number of additional classes required.

Case Study 2: Retail Billing with Discount

1. Problem Statement:

Calculate the bill for multiple products by multiplying each product's price by its quantity, applying a discount when the subtotal reaches a specified amount, and adding tax.

2. Objectives:

- Calculate the cost of each product.
- Apply a discount based on the subtotal.
- Calculate tax and the final bill.

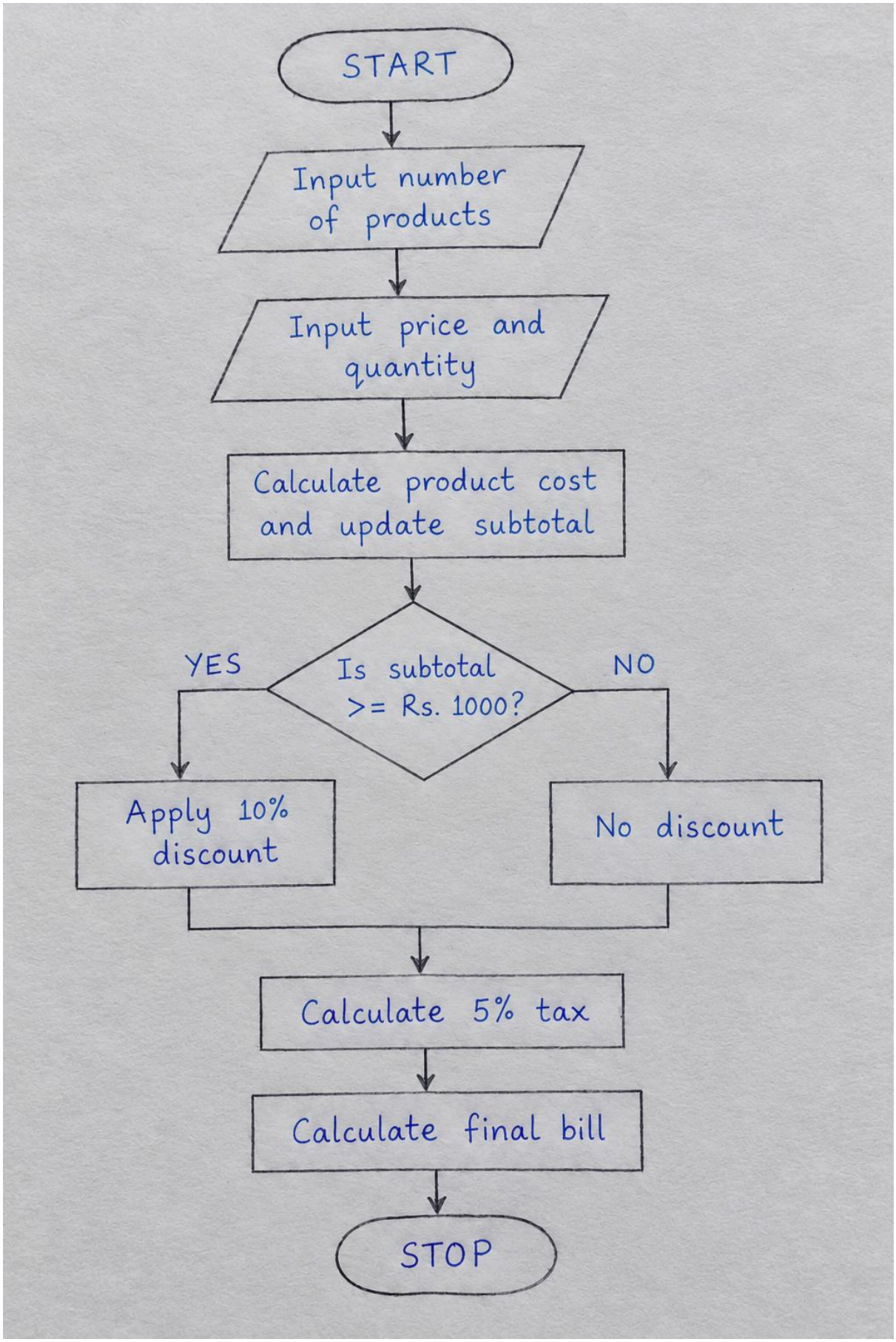
3. Assumptions:

- Prices and quantities must be non-negative.
- A 10% discount applies when the subtotal is at least ₹1,000.
- Tax is 5% of the discounted subtotal.

4. Algorithm:

1. Start.
2. Input the number of products.
3. Input the price and quantity of each product.
4. Calculate the subtotal using price × quantity.
5. If the subtotal is at least ₹1,000, apply a 10% discount.
6. Calculate 5% tax on the discounted amount.
7. Calculate the final bill.
8. Display subtotal, discount, tax, and final bill.
9. Stop.

5. Flowchart:


START
Input number of products
Input price and quantity
Calculate product cost and update subtotal
Is subtotal >= Rs. 1000?
YES
NO
Apply 10% discount
No discount
Calculate 5% tax
Calculate final bill
STOP

6. Pseudocode:

START

INPUT n

subtotal = 0

REPEAT n TIMES

  INPUT price, quantity

  subtotal = subtotal + price * quantity

IF subtotal >= 1000

  discount = subtotal * 0.10

ELSE

  discount = 0

taxable_amount = subtotal - discount

tax = taxable_amount * 0.05

total = taxable_amount + tax

DISPLAY subtotal, discount, tax, total

stop

7. Python Source Code:

```
n = int(input("Number of products: "))
subtotal = 0

for i in range(n):
  price = float(input("Enter product price: "))
  quantity = int(input("Enter quantity: "))
```

```
    if price < 0 or quantity < 0:
        print("Invalid product details")
        break

    subtotal += price * quantity

else:
    if subtotal >= 1000:
        discount = subtotal * 0.10
    else:
        discount = 0

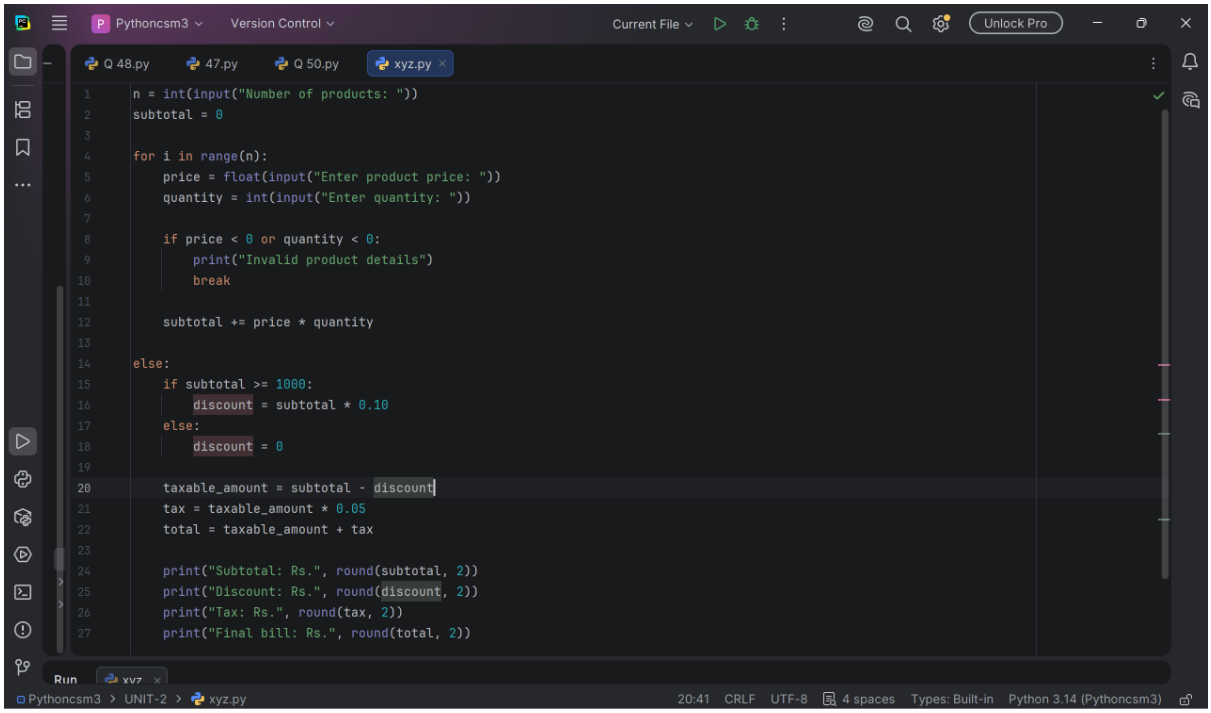
    taxable_amount = subtotal - discount
    tax = taxable_amount * 0.05
    total = taxable_amount + tax

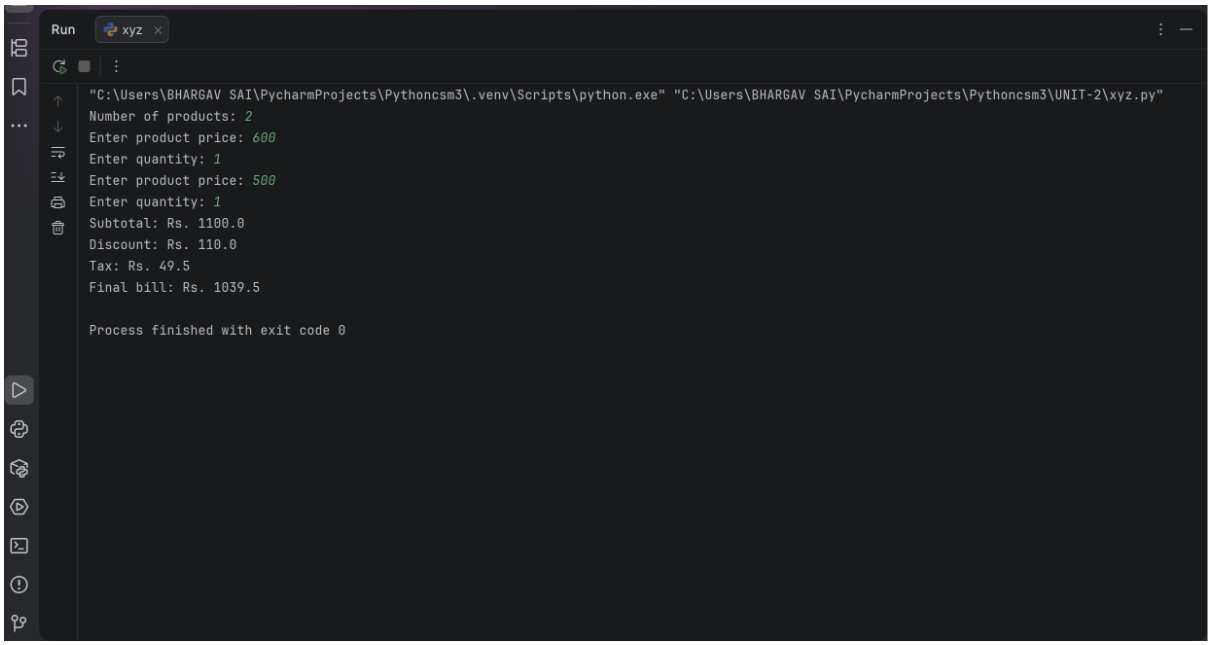
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("Tax: Rs.", round(tax, 2))
    print("Final bill: Rs.", round(total, 2))
```

8. Test Cases:

Product details	Expected result
₹500 × 1	Subtotal ₹500; total ₹525
₹600 × 1 and ₹500 × 1	Subtotal ₹1,100; total ₹1,039.50
Negative product price	Invalid product details

9. Output:





10. Conclusion:

Loops and conditional statements help create a billing system that calculates discounts and taxes automatically.

Case Study 3: ATM Transaction Sequence Simulation

1. Problem Statement:

Simulate a sequence of bank deposits and withdrawals, update the account balance, reject withdrawals that exceed the available balance, and display the final balance.

2. Objectives:

- Process multiple transactions.
- Update the account balance.
- Reject invalid transactions.
- Display the final balance.

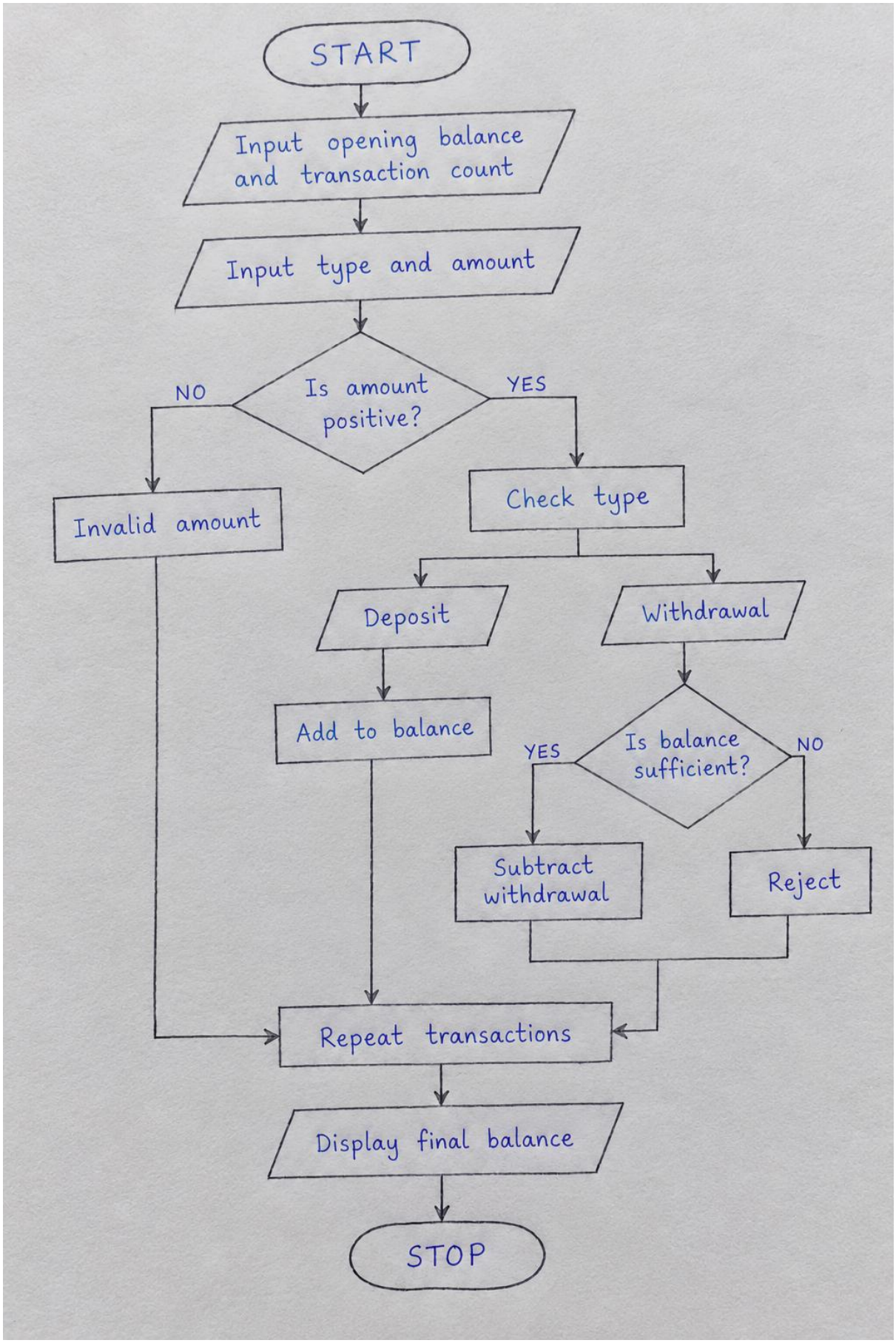
3. Assumptions:

- The opening balance is non-negative.
- D represents a deposit.
- W represents a withdrawal.
- Transaction amounts must be positive.

4. Algorithm:

1. Start.
2. Input the opening balance.
3. Input the number of transactions.
4. Repeat for each transaction:
    - Input transaction type and amount.
    - Check whether the amount is positive.
    - If the type is D, add the amount to the balance.
    - If the type is W, check whether the balance is sufficient.
    - Reject the withdrawal if the balance is insufficient.
5. Display the final balance.
6. Stop.

5. Flowchart:


START
Input opening balance and transaction count
Input type and amount
Is amount positive?
NO
YES
Invalid amount
Check type
Deposit
Withdrawal
Add to balance
Is balance sufficient?
YES
NO
Subtract withdrawal
Reject
Repeat transactions
Display final balance
STOP

6. Pseudocode:

START

INPUT balance, n

REPEAT n TIMES

  INPUT type, amount

  IF amount <= 0

    DISPLAY "Invalid amount"

  ELSE IF type == "D"

    balance = balance + amount

  ELSE IF type == "W" AND amount <= balance

    balance = balance - amount

  ELSE IF type == "W"

    DISPLAY "Insufficient balance"

  ELSE

    DISPLAY "Invalid transaction type"

DISPLAY balance

STOP

7. Python Source Code:

```
balance = float(input("Opening balance: Rs. "))
n = int(input("Number of transactions: "))
```

```
for i in range(n):
    kind = input("Transaction type (D/W): ").upper()
    amount = float(input("Amount: Rs. "))

    if amount <= 0:
        print("Invalid amount")

    elif kind == "D":
        balance += amount
        print("Deposit successful")

    elif kind == "W":
        if amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance")

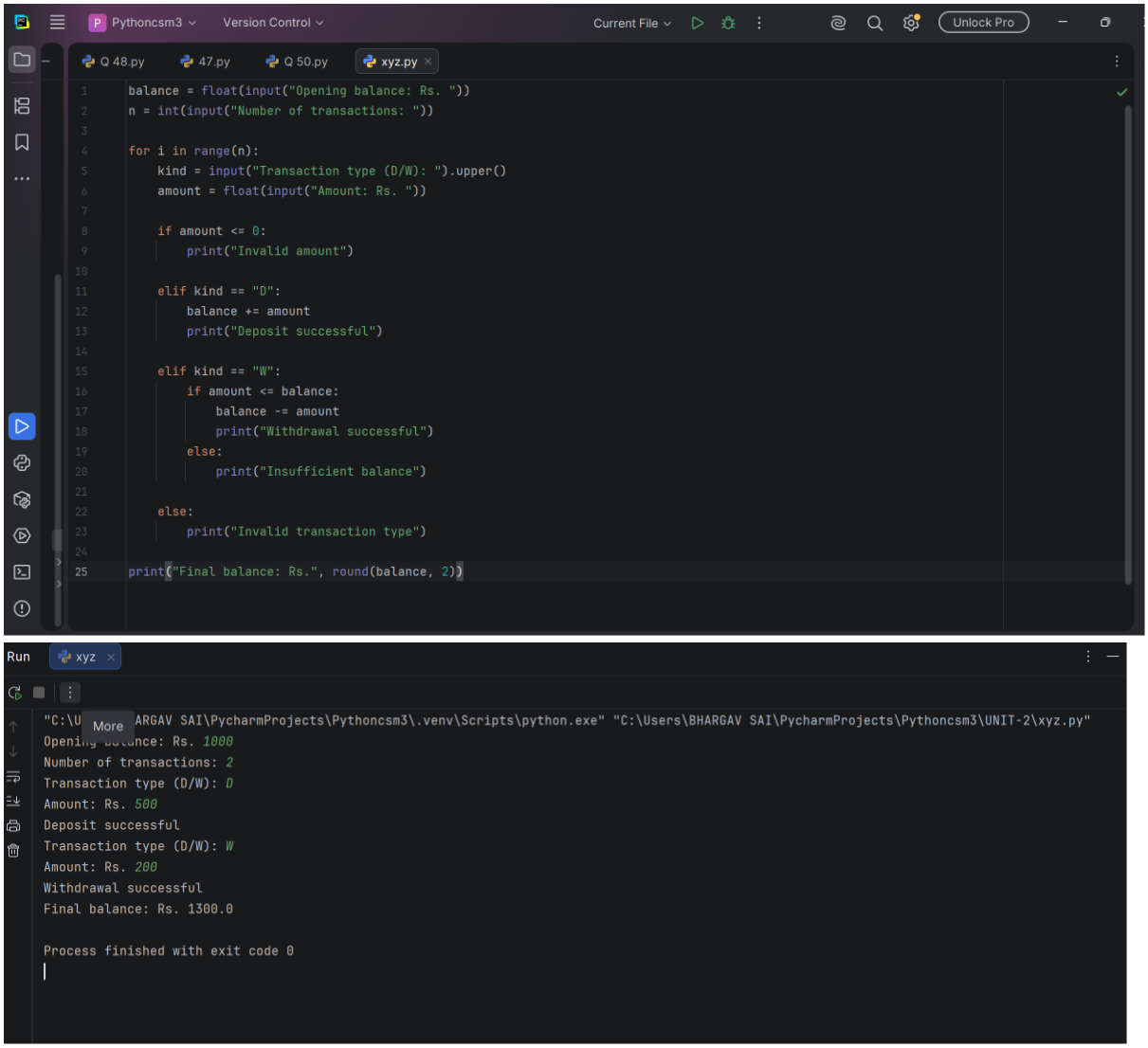
    else:
        print("Invalid transaction type")

print("Final balance: Rs.", round(balance, 2))
```

8. Test Cases:

Opening balance	Transactions	Expected result
₹1,000	Deposit ₹500; withdraw ₹200	Final balance ₹1,300
₹300	Withdraw ₹500	Withdrawal rejected; balance ₹300
₹100	Deposit −₹20	Invalid amount; balance ₹100

9. Output:



10. Conclusion:

A simple ATM simulation demonstrates loops, conditional statements, and updating a running balance.

Case Study 4: Warehouse Inventory Reconciliation

1. Problem Statement:

Calculate the expected closing stock of a product using opening stock, received stock, and dispatched stock. Compare it with the physical stock count to identify shortages or surpluses.

2. Objectives:

- Calculate expected closing stock.
- Compare expected and physical stock.
- Identify shortages and surpluses.

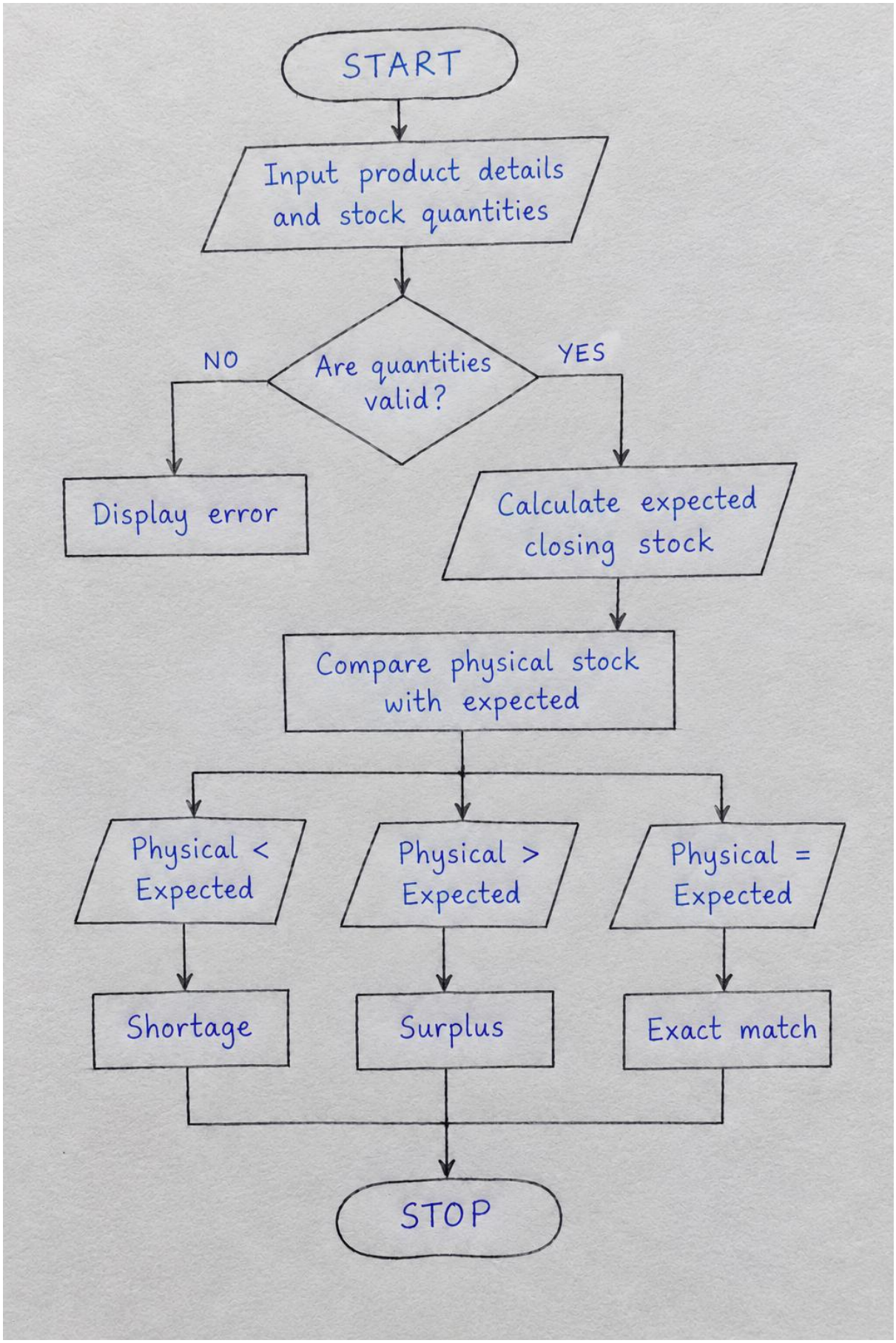
3. Assumptions:

- Stock quantities are non-negative whole numbers.
- Expected closing stock = opening stock + received stock − dispatched stock.
- Physical stock is counted after dispatch.

4. Algorithm:

1. Start.
2. Input the product name.
3. Input opening stock, received stock, dispatched stock, and physical stock.
4. Validate all quantities.
5. Check that dispatched stock does not exceed available stock.
6. Calculate expected closing stock.
7. Compare expected stock with physical stock.
8. Display a shortage, surplus, or exact match.
9. Stop.

5. Flowchart:


START
Input product details and stock quantities
Are quantities valid?
NO
YES
Display error
Calculate expected closing stock
Compare physical stock with expected
Physical < Expected
Physical > Expected
Physical = Expected
Shortage
Surplus
Exact match
STOP

6. Pseudocode:

START

INPUT product, opening, received, dispatched, physical

IF any quantity is negative

  DISPLAY "Invalid stock quantity"

ELSE IF dispatched > opening + received

  DISPLAY "Dispatch exceeds available stock"

ELSE

  expected = opening + received - dispatched

  difference = physical - expected

  IF difference < 0

    DISPLAY shortage = -difference

  ELSE IF difference > 0

    DISPLAY surplus = difference

  ELSE

    DISPLAY "Stock matches exactly"

STOP

7. Python Source Code:

```
product = input("Product name: ")
opening = int(input("Opening stock: "))
received = int(input("Stock received: "))
dispatched = int(input("Stock dispatched: "))
physical = int(input("Physical stock count: "))
```

```
if min(opening, received, dispatched, physical) < 0:
    print("Invalid stock quantity")

elif dispatched > opening + received:
    print("Invalid: dispatch exceeds available stock")

else:
    expected = opening + received - dispatched
    difference = physical - expected

    print("Product:", product)
    print("Expected closing stock:", expected)

    if difference < 0:
        print("Shortage:", -difference)

    elif difference > 0:
        print("Surplus:", difference)

    else:
        print("Stock matches exactly")
```

8. Test Cases:

Opening	Received	Dispatched	Physical	Expected result
100	20	30	90	Exact match
100	20	30	85	Shortage of 5
100	20	30	95	Surplus of 5

9. Output:

Q 48.py | 47.py | Q 50.py | xyz.py

```python
product = input("Product name: ")
opening = int(input("Opening stock: "))
received = int(input("Stock received: "))
dispatched = int(input("Stock dispatched: "))
physical = int(input("Physical stock count: "))

if min(opening, received, dispatched, physical) < 0:
    print("Invalid stock quantity")

elif dispatched > opening + received:
    print("Invalid: dispatch exceeds available stock")

else:
    expected = opening + received - dispatched
    difference = physical - expected

    print("Product:", product)
    print("Expected closing stock:", expected)

    if difference < 0:
        print("Shortage:", -difference)

    elif difference > 0:
        print("Surplus:", difference)

    else:
        print("Stock matches exactly")
```

```
"C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\.venv\Scripts\python.exe" "C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\UNIT-2\xyz.py"
Product name: note book
Opening stock: 100
Stock received: 20
Stock dispatched: 30
Physical stock count: 85
Product: note book
Expected closing stock: 90
Shortage: 5

Process finished with exit code 0
```

1o. Conclusion:

Stock reconciliation helps identify missing or extra items in a warehouse.

Case Study 5: Student Performance Trend Analysis

1. Problem Statement:

Enter a student's marks from several examinations, calculate the average, and determine whether the student's performance improved, declined, or remained unchanged between the first and last examinations.

2. Objectives:

- Store examination marks in a list.
- Calculate the average marks.
- Identify the performance trend.

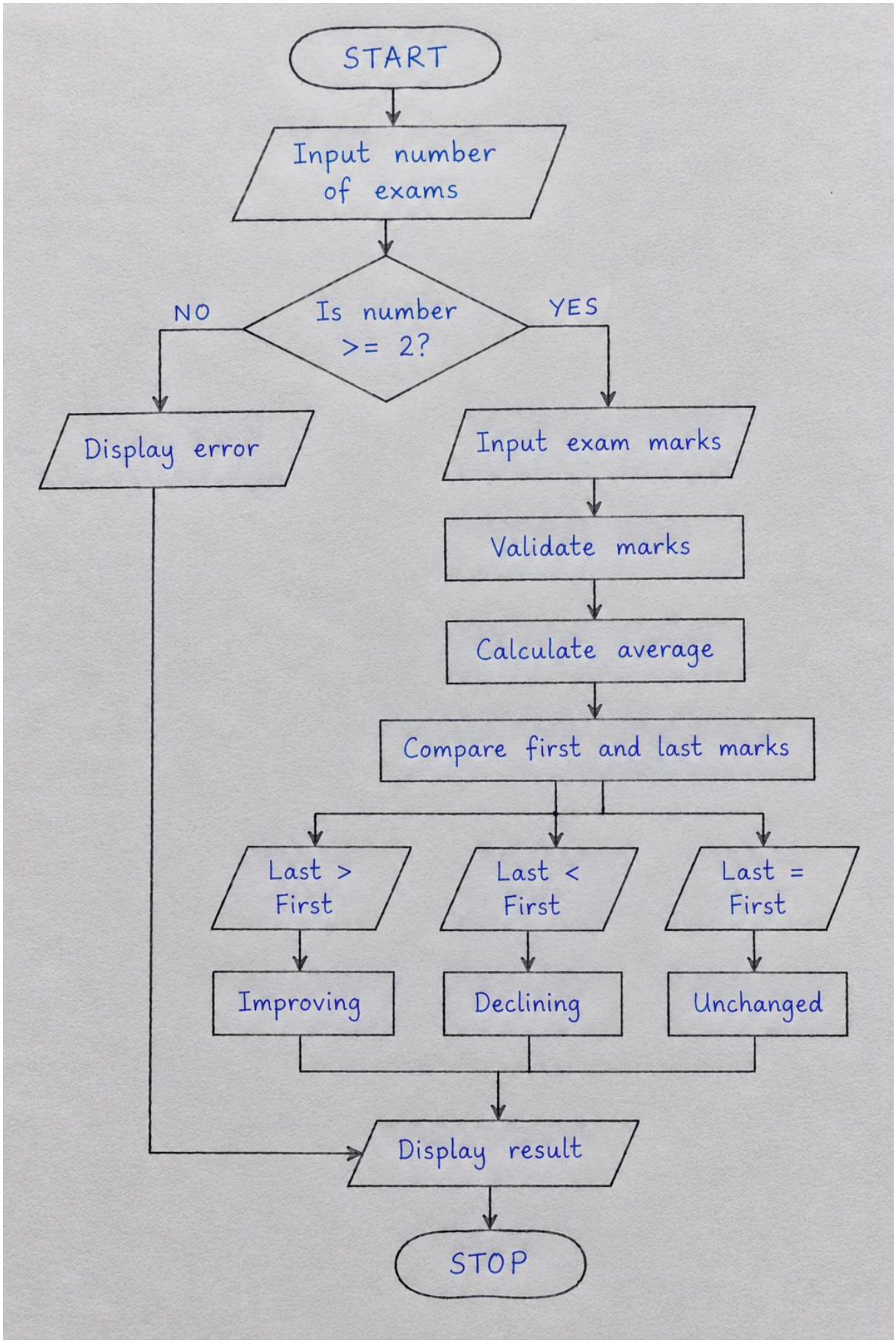
3. Assumptions:

- Marks must be between 0 and 100.
- At least two examination scores must be entered.
- The trend is based on the first and last marks.

4. Algorithm:

1. Start.
2. Input the number of examinations.
3. Check that at least two examinations are entered.
4. Input the marks and validate them.
5. Calculate the average marks.
6. Compare the first and last marks.
7. Display the average and performance trend.
8. Stop.

5. Flowchart:


START
Input number of exams
Is number >= 2?
NO
YES
Display error
Input exam marks
Validate marks
Calculate average
Compare first and last marks
Last > First
Last < First
Last = First
Improving
Declining
Unchanged
Display result
STOP

6. Pseudocode:

```
START
INPUT n

IF n < 2
  DISPLAY "Enter at least two examinations"
ELSE
  INPUT n marks into list

  IF any mark < 0 OR mark > 100
    DISPLAY "Invalid marks"
  ELSE
    average = SUM(marks) / n

    IF last mark > first mark
      trend = "Improving"
    ELSE IF last mark < first mark
      trend = "Declining"
    ELSE
      trend = "Unchanged"

    DISPLAY average, trend

STOP
```

7. Python Source Code:

```
n = int(input("Number of examinations (at least 2): "))

if n < 2:
```

```
    print("Enter at least two examinations")

else:
    marks = []
    valid = True

    for i in range(n):
        mark = float(input("Enter mark (0-100): "))

        if mark < 0 or mark > 100:
            valid = False

        marks.append(mark)

    if not valid:
        print("Invalid marks entered")

    else:
        average = sum(marks) / len(marks)

        if marks[-1] > marks[0]:
            trend = "Improving"

        elif marks[-1] < marks[0]:
            trend = "Declining"

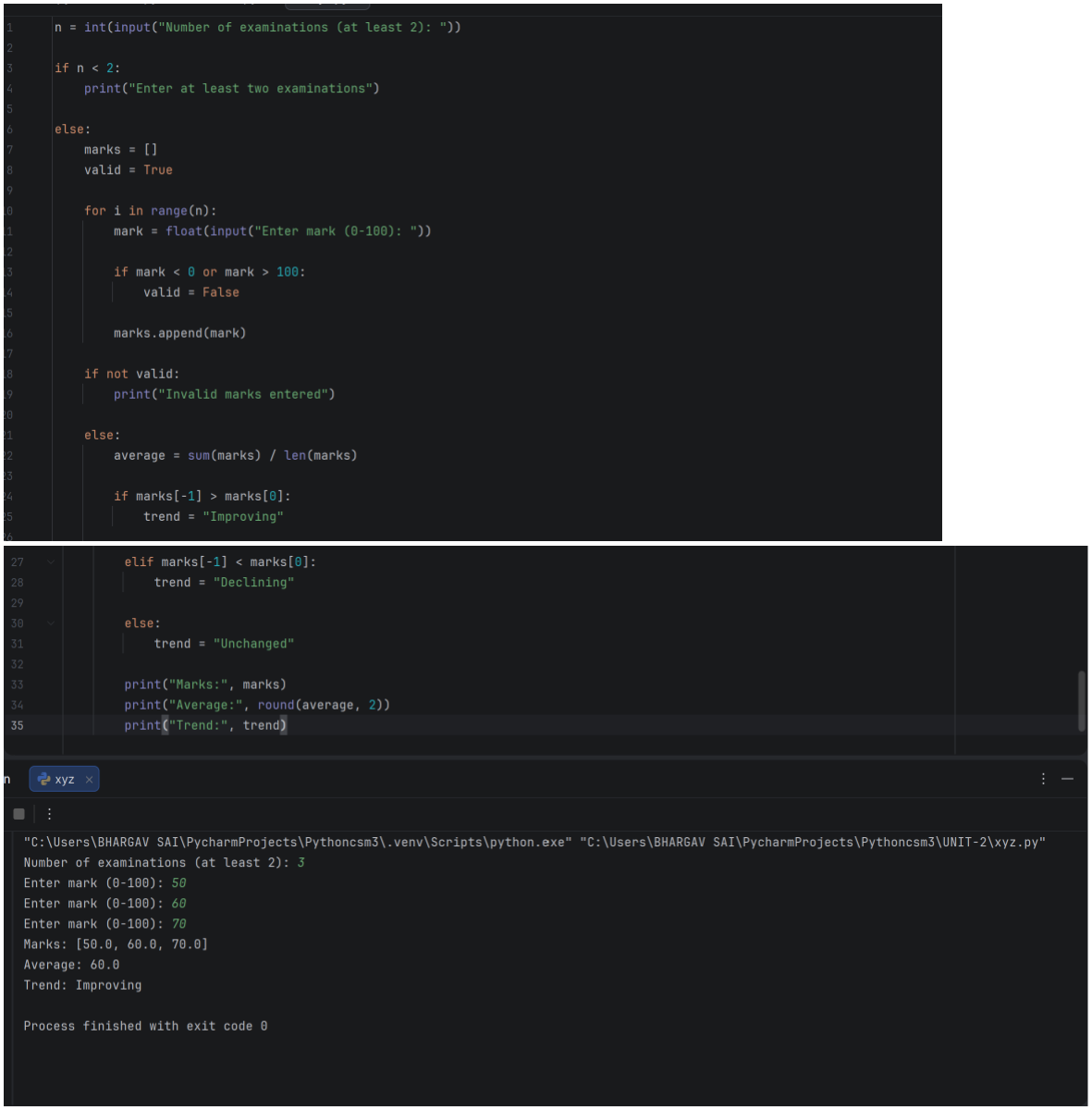
        else:
            trend = "Unchanged"
```

```
print("Marks:", marks)
print("Average:", round(average, 2))
print("Trend:", trend)
```

8. Test Cases:

Marks	Expected average	Expected trend
50, 60, 70	60	Improving
80, 70, 60	70	Declining
65, 70, 65	66.67	Unchanged

9. Output:



```
n = int(input("Number of examinations (at least 2): "))

if n < 2:
    print("Enter at least two examinations")

else:
    marks = []
    valid = True

    for i in range(n):
        mark = float(input("Enter mark (0-100): "))

        if mark < 0 or mark > 100:
            valid = False

        marks.append(mark)

    if not valid:
        print("Invalid marks entered")

    else:
        average = sum(marks) / len(marks)

        if marks[-1] > marks[0]:
            trend = "Improving"
        elif marks[-1] < marks[0]:
            trend = "Declining"

        else:
            trend = "Unchanged"

        print("Marks:", marks)
        print("Average:", round(average, 2))
        print("Trend:", trend)
```

```
xyz
"C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\.venv\Scripts\python.exe" "C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\UNIT-2\xyz.py"
Number of examinations (at least 2): 3
Enter mark (0-100): 50
Enter mark (0-100): 60
Enter mark (0-100): 70
Marks: [50.0, 60.0, 70.0]
Average: 60.0
Trend: Improving

Process finished with exit code 0
```

10. Conclusion:

Lists and loops help store and analyse student marks across multiple examinations.

Case Study 6: Food Waste Monitoring System

1. Problem Statement:

Compare meals prepared, meals served, and leftover meals across several days. Calculate the percentage of food wasted and identify days when waste exceeds the acceptable limit.

2. Objectives:

- Calculate leftover meals.
- Calculate the waste percentage.
- Identify days exceeding the waste limit.

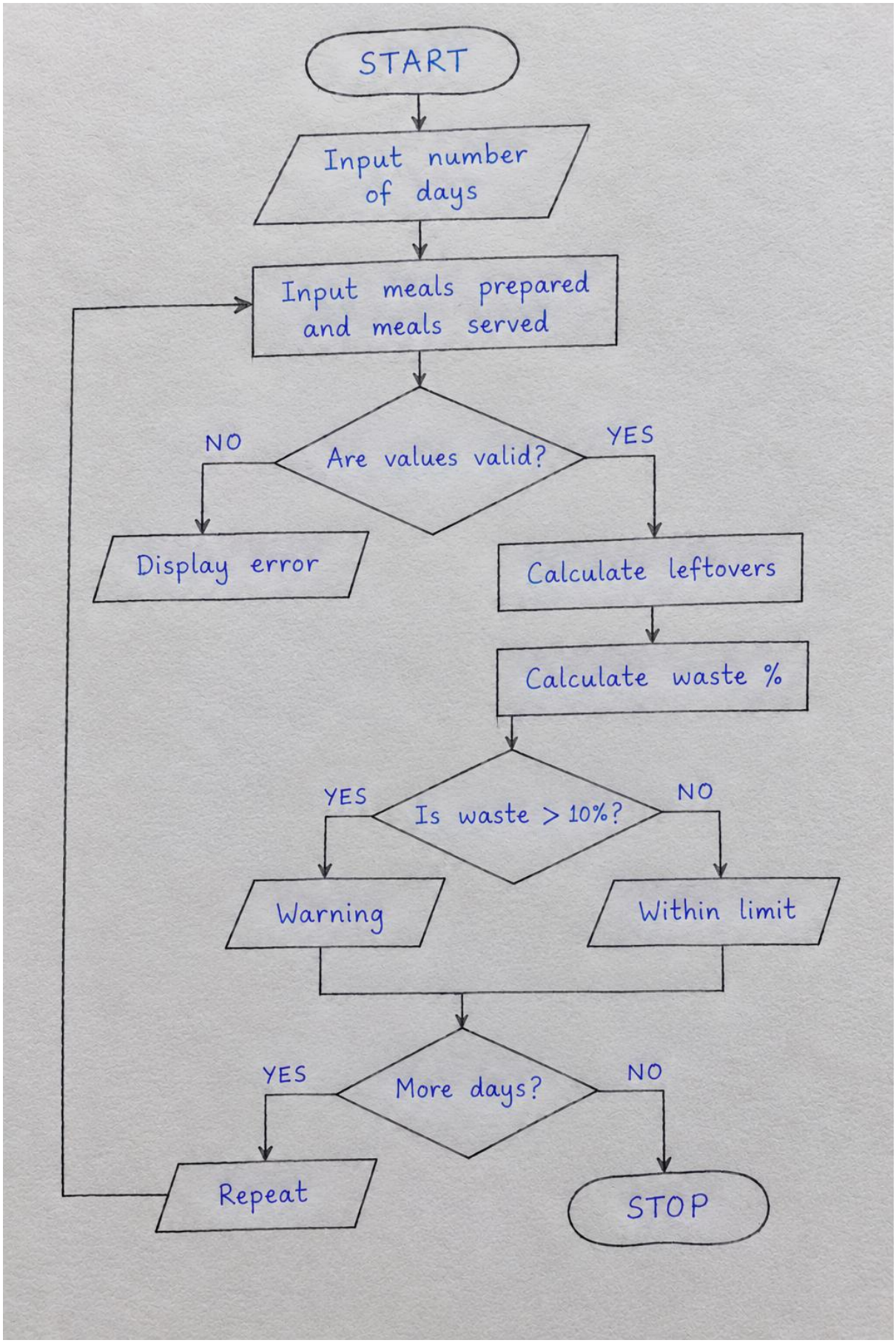
3. Assumptions:

- Meals prepared must be greater than zero.
- Meals served must be between zero and meals prepared.
- The acceptable waste limit is 10%.

4. Algorithm:

1. Start.
2. Input the number of days.
3. For each day, input meals prepared and meals served.
4. Validate the meal counts.
5. Calculate leftovers by subtracting meals served from meals prepared.
6. Calculate the waste percentage.
7. If waste exceeds 10%, display a warning.
8. Repeat for all days.
9. Stop.

5. Flowchart


START
Input number of days
Input meals prepared and meals served
Are values valid?
NO
YES
Display error
Calculate leftovers
Calculate waste %
Is waste > 10%?
YES
NO
Warning
Within limit
More days?
YES
NO
Repeat
STOP

6. Pseudocode:

START

INPUT days

FOR each day

  INPUT prepared, served

  IF prepared <= 0 OR served < 0 OR served > prepared

    DISPLAY "Invalid meal counts"

  ELSE

    leftovers = prepared - served

    waste = leftovers / prepared * 100

    DISPLAY leftovers, waste

    IF waste > 10

      DISPLAY "Warning: Waste is above 10%"

    ELSE

      DISPLAY "Waste is within the limit"

STOP

7. Python Source Code:

```
days = int(input("Number of days: "))

for day in range(1, days + 1):
  print("Day", day)

  prepared = int(input("Meals prepared: "))
```

```
served = int(input("Meals served: "))

if prepared <= 0 or served < 0 or served > prepared:
    print("Invalid meal counts")

else:
    leftovers = prepared - served
    waste = leftovers / prepared * 100

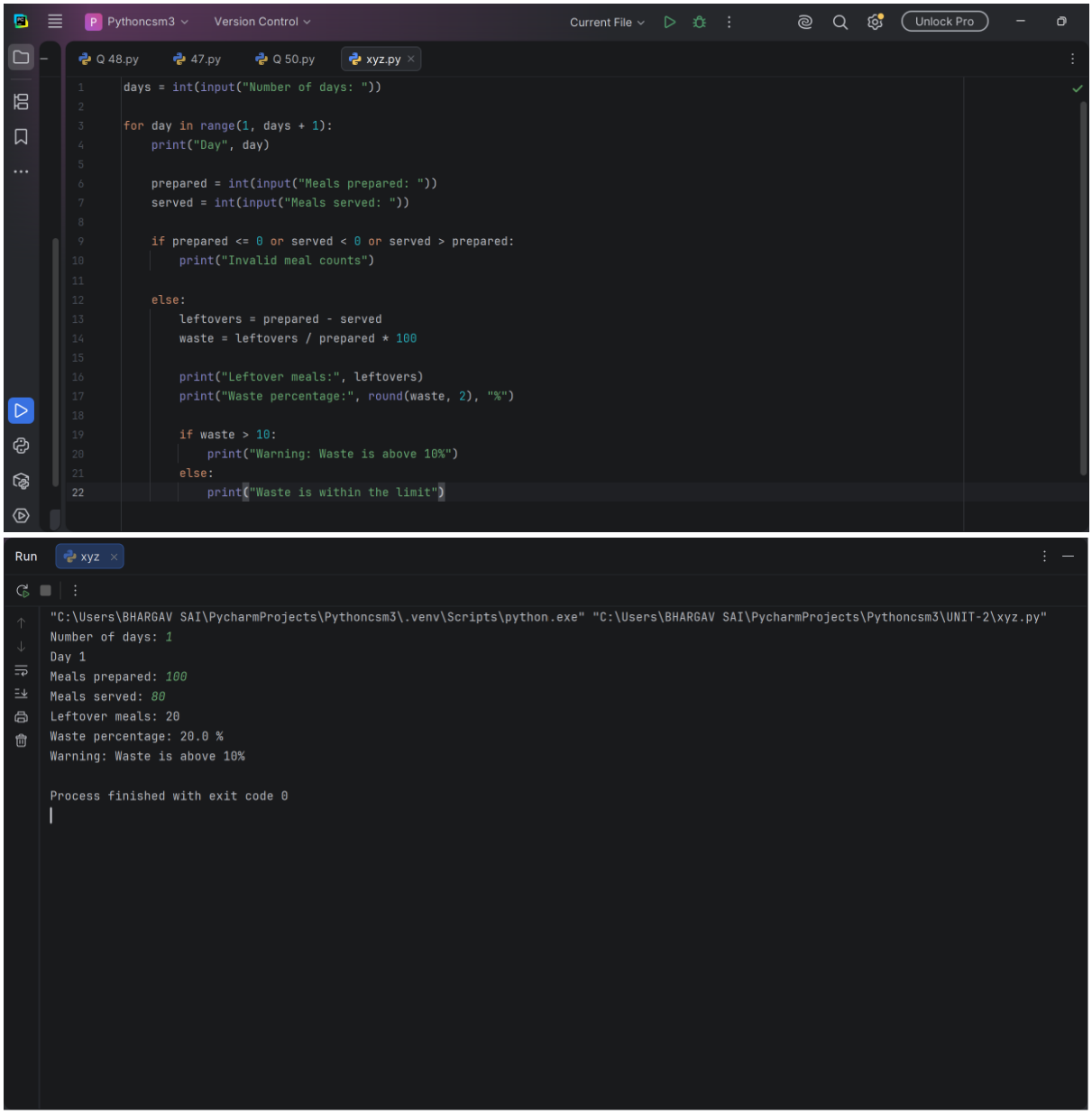
    print("Leftover meals:", leftovers)
    print("Waste percentage:", round(waste, 2), "%")

    if waste > 10:
        print("Warning: Waste is above 10%")
    else:
        print("Waste is within the limit")
```

8. Test Cases:

Meals prepared	Meals served	Expected result
100	95	5 leftovers; 5% waste
100	80	20 leftovers; 20% waste warning
50	60	Invalid meal counts

9. Output:



```
days = int(input("Number of days: "))

for day in range(1, days + 1):
    print("Day", day)

    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))

    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid meal counts")

    else:
        leftovers = prepared - served
        waste = leftovers / prepared * 100

        print("Leftover meals:", leftovers)
        print("Waste percentage:", round(waste, 2), "%")

        if waste > 10:
            print("Warning: Waste is above 10%")
        else:
            print("Waste is within the limit")
```

```
"C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\.venv\Scripts\python.exe" "C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\UNIT-2\xyz.py"
Number of days: 1
Day 1
Meals prepared: 100
Meals served: 80
Leftover meals: 20
Waste percentage: 20.0 %
Warning: Waste is above 10%

Process finished with exit code 0
```

10. Conclusion:

Basic calculations and conditional statements can help monitor food waste and encourage better food management.

Case Study 7: Progressive Electricity Billing

1. Problem Statement:

Calculate an electricity bill using different consumption slabs and a fixed charge.

2. Objectives:

- Calculate electricity charges using multiple slabs.
- Add a fixed charge.
- Display the total electricity bill.

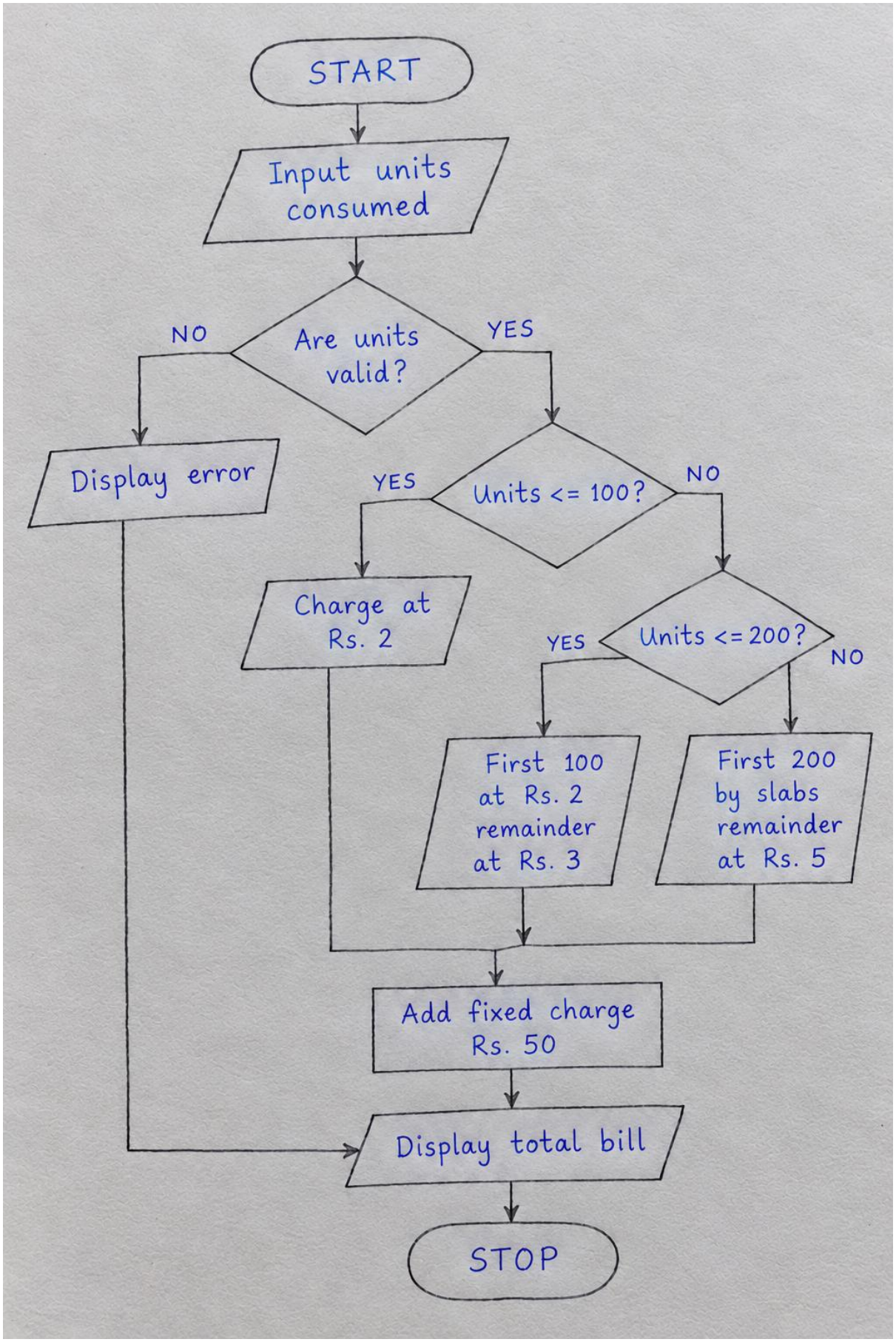
3. Assumptions:

- Units consumed cannot be negative.
- First 100 units cost ₹2 per unit.
- Next 100 units cost ₹3 per unit.
- Units above 200 cost ₹5 per unit.
- A fixed charge of ₹50 is added.

4. Algorithm:

1. Start.
2. Input the electricity units consumed.
3. If units are negative, display invalid input.
4. If units are up to 100, calculate charges at ₹2 per unit.
5. If units are between 101 and 200, calculate the first 100 units at ₹2 and remaining units at ₹3.
6. If units exceed 200, calculate the first two slabs and charge ₹5 for units above 200.
7. Add the fixed charge of ₹50.
8. Display the total bill.
9. Stop.

5. Flowchart:


START
Input units consumed
Are units valid?
NO
YES
Display error
Units <= 100?
YES
NO
Charge at Rs. 2
Units <= 200?
YES
NO
First 100 at Rs. 2 remainder at Rs. 3
First 200 by slabs remainder at Rs. 5
Add fixed charge Rs. 50
Display total bill
STOP

6. Pseudocode:

```
START

INPUT units


IF units < 0

   DISPLAY "Invalid units"

ELSE

   IF units <= 100

      energy = units * 2

   ELSE IF units <= 200

      energy = 100 * 2 + (units - 100) * 3

   ELSE

      energy = 100 * 2 + 100 * 3

            + (units - 200) * 5


   total = energy + 50


   DISPLAY energy, total

STOP
```

7. Python Source Code:

```
units = int(input("Electricity units consumed: "))


if units < 0:

   print("Invalid units")


else:

   if units <= 100:

      energy_charge = units * 2
```

```
elif units <= 200:
    energy_charge = 100 * 2 + (units - 100) * 3

else:
    energy_charge = (
        100 * 2
        + 100 * 3
        + (units - 200) * 5
    )

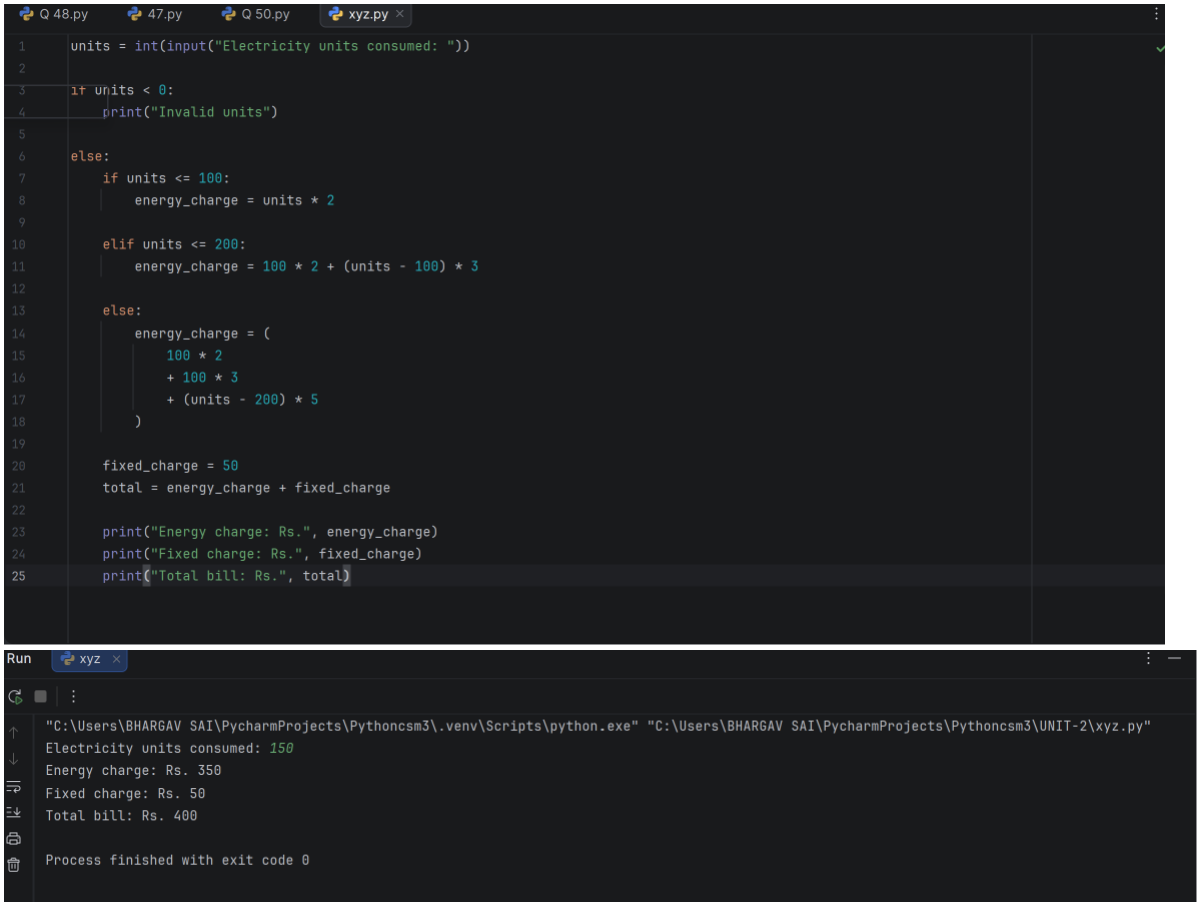
fixed_charge = 50
total = energy_charge + fixed_charge

print("Energy charge: Rs.", energy_charge)
print("Fixed charge: Rs.", fixed_charge)
print("Total bill: Rs.", total)
```

8. Test Cases:

Units consumed	Energy charge	Total bill
50	₹100	₹150
150	₹350	₹400
250	₹750	₹800

9. Output:



10. Conclusion:

Conditional statements make it possible to calculate electricity bills using progressive tariff slabs.

Case Study 8: Examination Result and Pass/Fail Validation

1. Problem Statement:

Calculate a student's total and average marks across five subjects and determine whether the student passes every subject based on the minimum pass mark.

2. Objectives:

- Store marks for multiple subjects.
- Calculate total and average marks.
- Check subject-wise pass criteria.
- Display the final result.

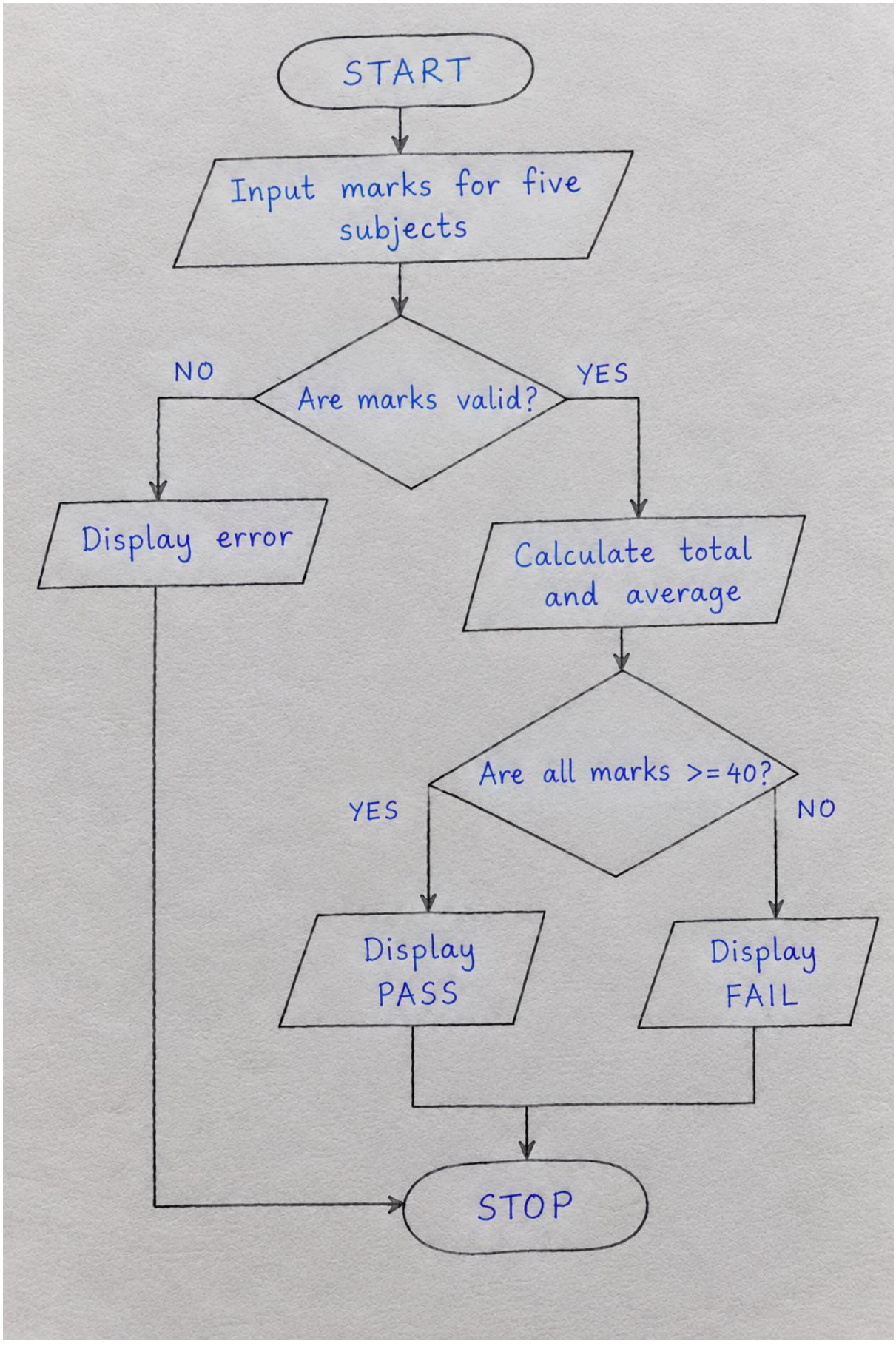
3. Assumptions:

- There are five subjects.
- Each subject is marked out of 100.
- The minimum pass mark is 40 in every subject.
- Marks must be between 0 and 100.

4. Algorithm:

1. Start.
2. Input marks for five subjects.
3. Validate that every mark is between 0 and 100.
4. Calculate the total marks.
5. Calculate the average marks.
6. Check whether every subject mark is at least 40.
7. Display the total, average, and pass/fail result.
8. Stop.

5. Flowchart:


START
Input marks for five subjects
Are marks valid?
NO
YES
Display error
Calculate total and average
Are all marks >= 40?
YES
NO
Display PASS
Display FAIL
STOP

6. Pseudocode:

```
START
marks = empty list

REPEAT 5 TIMES
  INPUT mark
  ADD mark to marks

IF any mark < 0 OR mark > 100
  DISPLAY "Invalid marks"
ELSE
  total = SUM(marks)
  average = total / 5
  passed_all = TRUE

  FOR each mark in marks
    IF mark < 40
      passed_all = FALSE

  DISPLAY total, average

  IF passed_all
    DISPLAY "PASS"
  ELSE
    DISPLAY "FAIL"

STOP
```

7. Python Source Code:

```
marks = []
valid = True

for i in range(1, 6):
    mark = float(input(
        "Enter marks for subject " + str(i) + ": "
    ))

    if mark < 0 or mark > 100:
        valid = False

    marks.append(mark)

if not valid:
    print("Invalid marks. Enter marks from 0 to 100.")

else:
    total = sum(marks)
    average = total / len(marks)
    passed_all = True

    for mark in marks:
        if mark < 40:
            passed_all = False

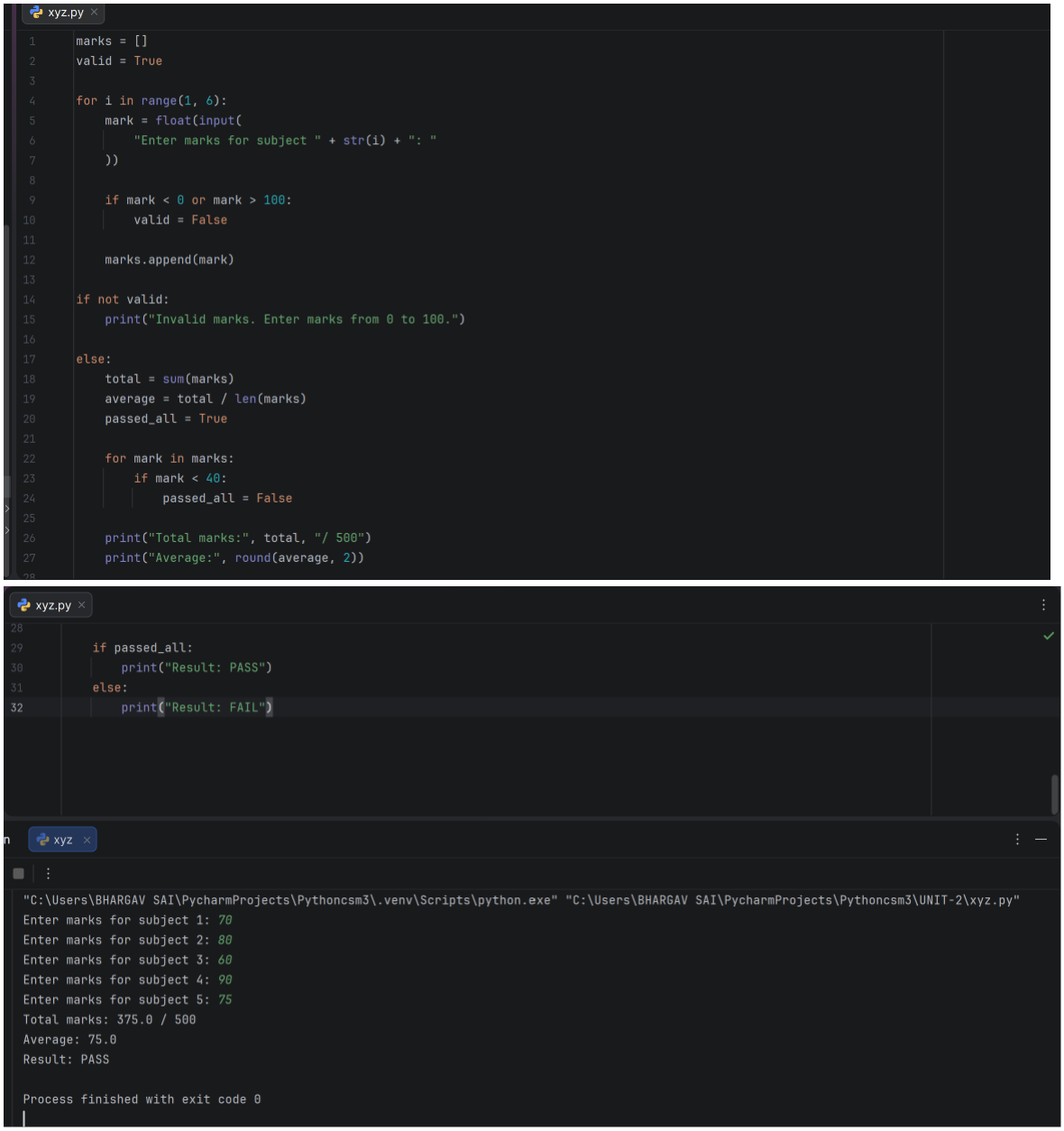
    print("Total marks:", total, "/ 500")
    print("Average:", round(average, 2))
```

```
if passed_all:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

8. Test Cases:

Subject marks	Total	Average	Result
70, 80, 60, 90, 75	375	75	PASS
70, 35, 60, 90, 75	330	66	FAIL
100, 100, 100, 100, 105	—	—	Invalid marks

9.Output:



```python
marks = []
valid = True

for i in range(1, 6):
    mark = float(input(
        "Enter marks for subject " + str(i) + ": "
    ))

    if mark < 0 or mark > 100:
        valid = False

    marks.append(mark)

if not valid:
    print("Invalid marks. Enter marks from 0 to 100.")

else:
    total = sum(marks)
    average = total / len(marks)
    passed_all = True

    for mark in marks:
        if mark < 40:
            passed_all = False

    print("Total marks:", total, "/ 500")
    print("Average:", round(average, 2))
```

```python
    if passed_all:
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

```
"C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\.venv\Scripts\python.exe" "C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\UNIT-2\xyz.py"
Enter marks for subject 1: 70
Enter marks for subject 2: 80
Enter marks for subject 3: 60
Enter marks for subject 4: 90
Enter marks for subject 5: 75
Total marks: 375.0 / 500
Average: 75.0
Result: PASS

Process finished with exit code 0
```

10. Conclusion:

Lists, loops, and conditional statements help process examination marks and apply pass criteria.

Case Study 9: Campus Event Financial Planning

1. Problem Statement:

Calculate total campus event expenses, compare them with sponsorship, and determine the minimum number of participants needed to cover the remaining cost through registration fees.

2. Objectives:

- Calculate total event expenses.
- Calculate the funding shortfall after sponsorship.
- Determine the minimum number of registrations needed to break even.

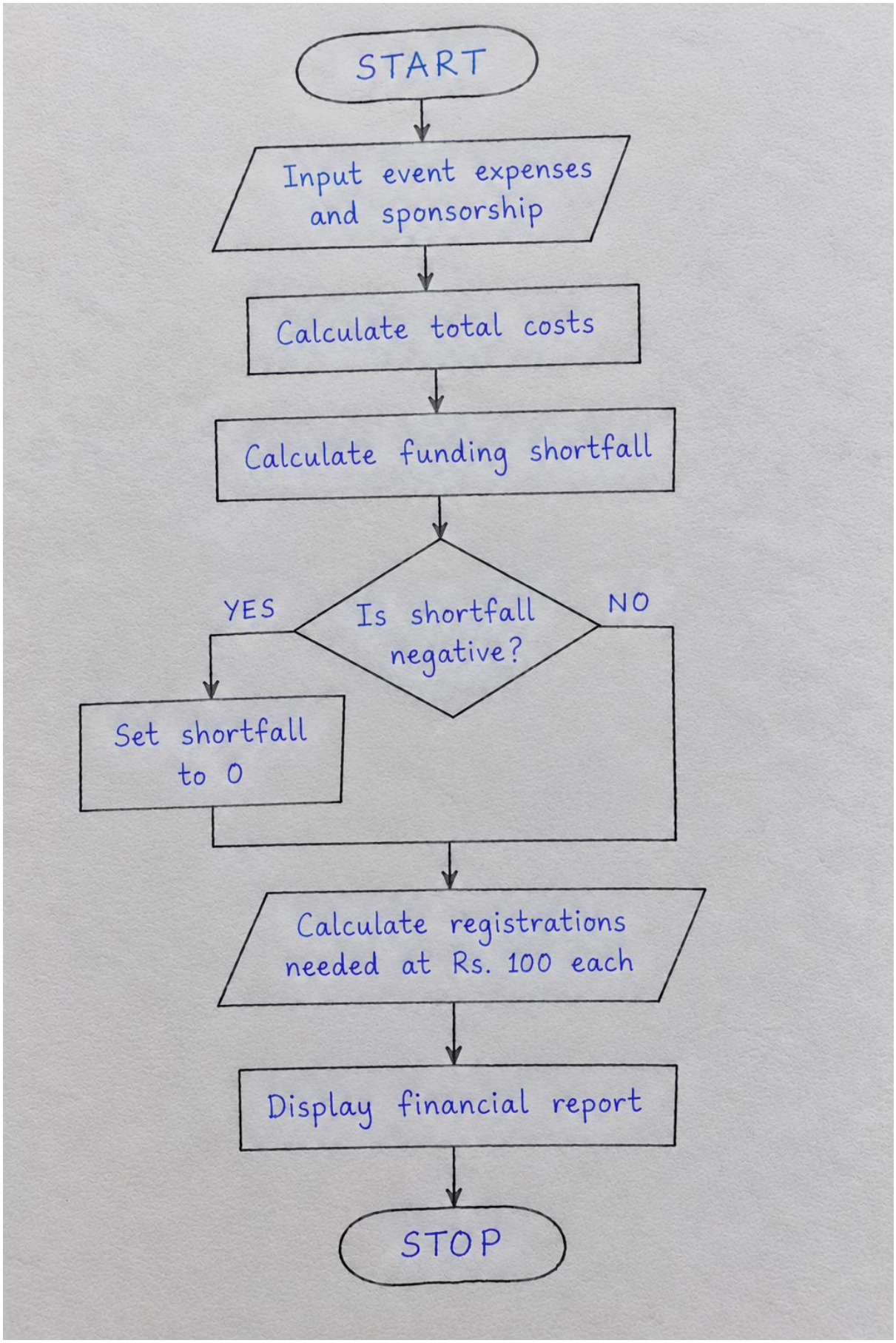
3. Assumptions:

- All costs and sponsorship amounts are non-negative.
- Each participant pays a registration fee of ₹100.
- The number of registrations must be a whole number.

4. Algorithm:

1. Start.
2. Input venue, equipment, catering, transport, and certificate costs.
3. Input the sponsorship amount.
4. Calculate total expenses.
5. Subtract sponsorship from total expenses.
6. If the shortfall is negative, set it to zero.
7. Use a loop to calculate the minimum registrations needed at ₹100 each.
8. Display total expenses, shortfall, and registrations needed.
9. Stop.

5. Flowchart:


START
Input event expenses and sponsorship
Calculate total costs
Calculate funding shortfall
Is shortfall negative?
YES
NO
Set shortfall to 0
Calculate registrations needed at Rs. 100 each
Display financial report
STOP

6. Pseudocode:

```
START
INPUT venue, equipment, catering
INPUT transport, certificates, sponsorship

expenses = venue + equipment + catering
        + transport + certificates

shortfall = expenses - sponsorship

IF shortfall < 0
   shortfall = 0

registrations = 0

WHILE registrations * 100 < shortfall
   registrations = registrations + 1

DISPLAY expenses, shortfall, registrations
STOP
```

7. Python Source Code:

```
venue = float(input("Venue cost: Rs. "))
equipment = float(input("Equipment cost: Rs. "))
catering = float(input("Catering cost: Rs. "))
transport = float(input("Transport cost: Rs. "))
certificates = float(input("Certificate cost: Rs. "))
sponsorship = float(input("Sponsorship amount: Rs. "))
```

```
if min(venue, equipment, catering, transport,
      certificates, sponsorship) < 0:
   print("Invalid cost")


else:
   expenses = (
      venue + equipment + catering
      + transport + certificates
   )


   shortfall = max(0, expenses - sponsorship)


   registrations = 0

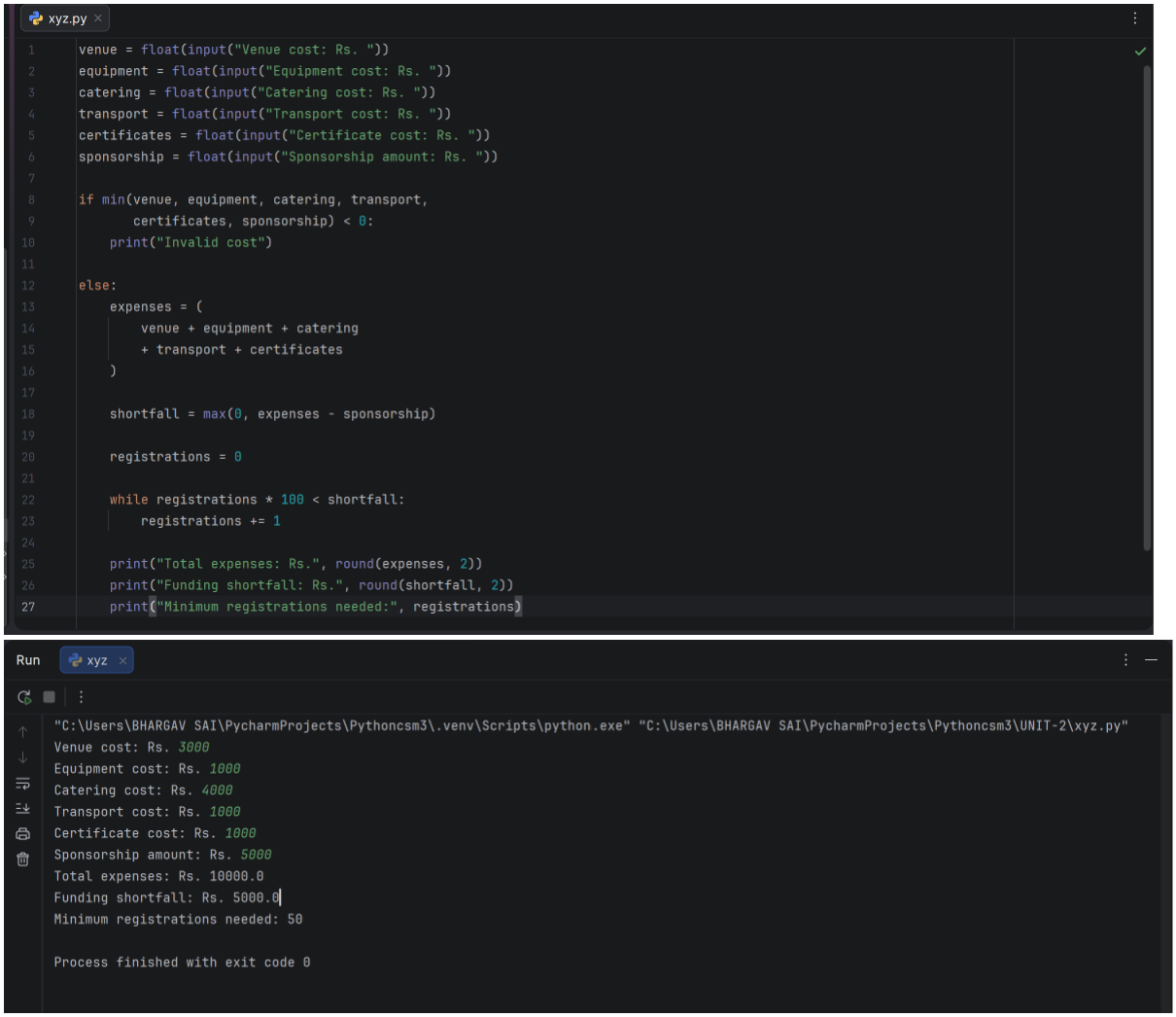

   while registrations * 100 < shortfall:
      registrations += 1


   print("Total expenses: Rs.", round(expenses, 2))
   print("Funding shortfall: Rs.", round(shortfall, 2))
   print("Minimum registrations needed:", registrations)
```

8. Test Cases:

Total expenses	Sponsorship	Shortfall	Registrations needed
₹10,000	₹5,000	₹5,000	50
₹8,000	₹9,000	₹0	0
₹10,000	₹0	₹10,000	100

9. Output:



```
xyz.py
venue = float(input("Venue cost: Rs. "))
equipment = float(input("Equipment cost: Rs. "))
catering = float(input("Catering cost: Rs. "))
transport = float(input("Transport cost: Rs. "))
certificates = float(input("Certificate cost: Rs. "))
sponsorship = float(input("Sponsorship amount: Rs. "))

if min(venue, equipment, catering, transport,
       certificates, sponsorship) < 0:
    print("Invalid cost")

else:
    expenses = (
        venue + equipment + catering
        + transport + certificates
    )

    shortfall = max(0, expenses - sponsorship)

    registrations = 0

    while registrations * 100 < shortfall:
        registrations += 1

    print("Total expenses: Rs.", round(expenses, 2))
    print("Funding shortfall: Rs.", round(shortfall, 2))
    print("Minimum registrations needed:", registrations)
```

```
Run  xyz
"C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\.venv\Scripts\python.exe" "C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\UNIT-2\xyz.py"
Venue cost: Rs. 3000
Equipment cost: Rs. 1000
Catering cost: Rs. 4000
Transport cost: Rs. 1000
Certificate cost: Rs. 1000
Sponsorship amount: Rs. 5000
Total expenses: Rs. 10000.0
Funding shortfall: Rs. 5000.0
Minimum registrations needed: 50

Process finished with exit code 0
```

10. Conclusion:

Loops and arithmetic operations can help plan event budgets and estimate registration requirements.

Case Study 10: Smart Canteen Order and Stock Management

1. Problem Statement:

Process multiple canteen orders, calculate their costs, update available stock, and reject orders that request more items than are available.

2. Objectives:

- Maintain the available stock.
- Calculate bills for customer orders.
- Reject unavailable quantities.
- Display total sales and remaining stock.

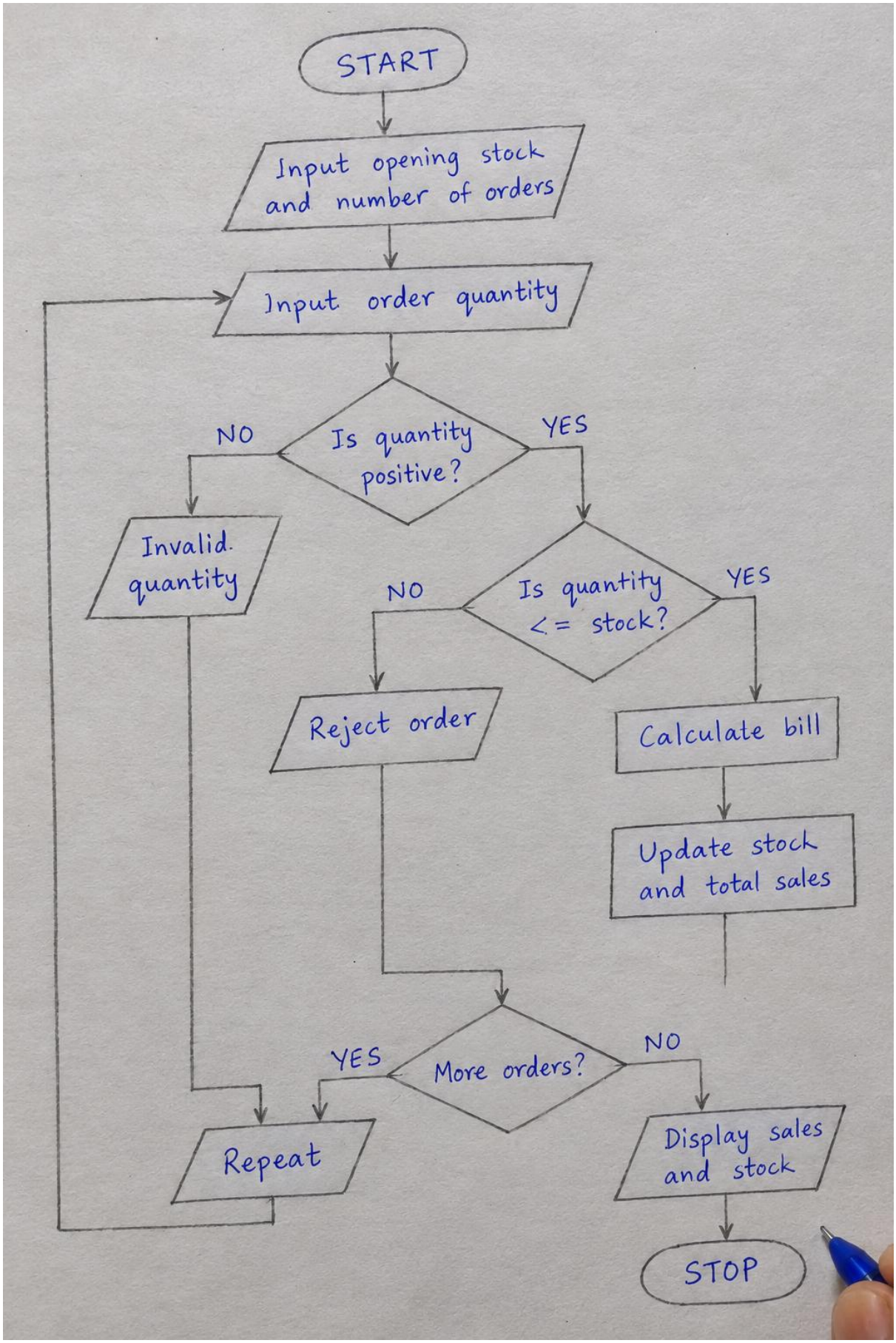
3. Assumptions:

- The canteen sells one type of item in this simple program.
- Each item costs ₹30.
- Order quantities must be positive.
- The opening stock is entered by the user.

4. Algorithm:

1. Start.
2. Input opening stock.
3. Input the number of orders.
4. Set total sales to zero.
5. For each order, input the requested quantity.
6. If the quantity is zero or negative, reject it.
7. If the quantity exceeds available stock, reject the order.
8. Otherwise, calculate the bill, update stock, and add the bill to total sales.
9. Display total sales and remaining stock.
10. Stop.

5. Flowchart:


START
Input opening stock and number of orders
Input order quantity
Is quantity positive?
NO
YES
Invalid quantity
Is quantity <= stock?
NO
YES
Reject order
Calculate bill
Update stock and total sales
More orders?
YES
NO
Repeat
Display sales and stock
STOP

6. Pseudocode:

```
START
INPUT stock, n
sales = 0
price = 30

REPEAT n TIMES
  INPUT quantity

  IF quantity <= 0
    DISPLAY "Invalid quantity"

  ELSE IF quantity > stock
    DISPLAY "Not enough stock"

  ELSE
    bill = quantity * price
    stock = stock - quantity
    sales = sales + bill
    DISPLAY bill

DISPLAY sales, stock
STOP
```

7. Python Source Code:

```
stock = int(input("Opening item stock: "))
n = int(input("Number of orders: "))

total_sales = 0
```

```
price = 30

for i in range(n):
    quantity = int(input("Order quantity: "))

    if quantity <= 0:
        print("Invalid quantity")

    elif quantity > stock:
        print("Order rejected: not enough stock")

    else:
        bill = quantity * price
        stock -= quantity
        total_sales += bill

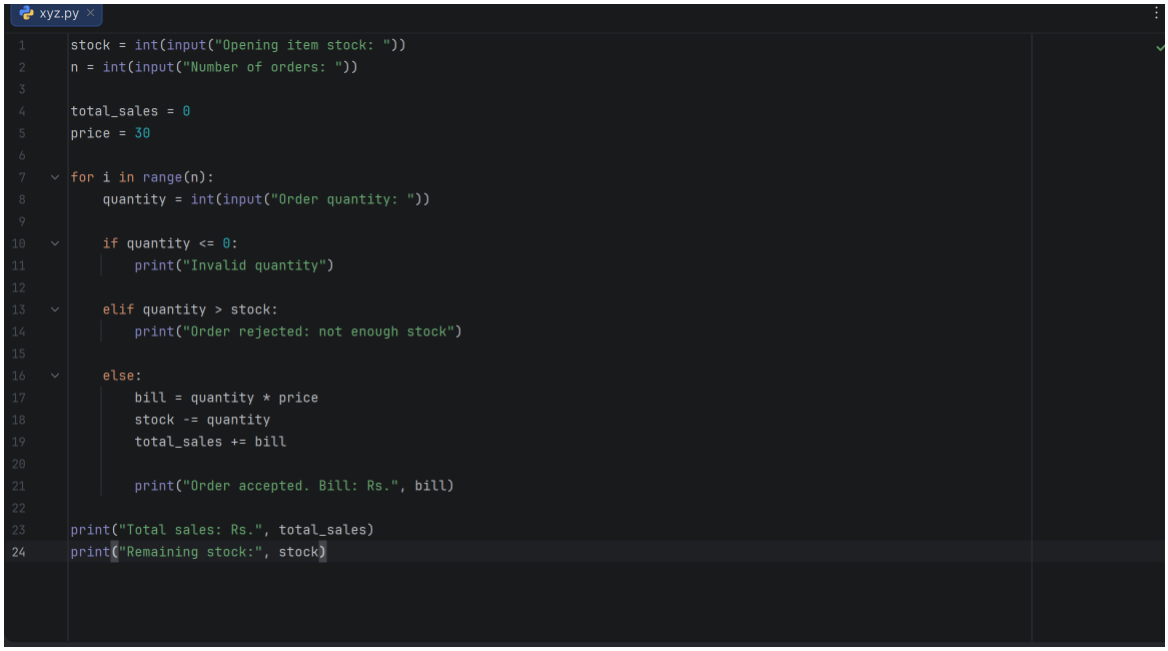
        print("Order accepted. Bill: Rs.", bill)

print("Total sales: Rs.", total_sales)
print("Remaining stock:", stock)
```

8. Test Cases:

Opening stock	Orders	Expected result
10	2 and 3	Sales ₹150; remaining stock 5
4	5	Order rejected; remaining stock 4
10	0	Invalid quantity; remaining stock 10

9. Output:



```python
stock = int(input("Opening item stock: "))
n = int(input("Number of orders: "))

total_sales = 0
price = 30

for i in range(n):
    quantity = int(input("Order quantity: "))

    if quantity <= 0:
        print("Invalid quantity")

    elif quantity > stock:
        print("Order rejected: not enough stock")

    else:
        bill = quantity * price
        stock -= quantity
        total_sales += bill

        print("Order accepted. Bill: Rs.", bill)

print("Total sales: Rs.", total_sales)
print("Remaining stock:", stock)
```



```
"C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\.venv\Scripts\python.exe" "C:\Users\BHARGAV SAI\PycharmProjects\Pythoncsm3\UNIT-2\xyz.py"
Opening item stock: 10
Number of orders: 2
Order quantity: 2
Order accepted. Bill: Rs. 60
Order quantity: 3
Order accepted. Bill: Rs. 90
Total sales: Rs. 150
Remaining stock: 5

Process finished with exit code 0
```

10. Conclusion:

Loops and conditional statements help process repeated customer orders while maintaining accurate stock records.